Student Name: Govinda Bist

Student Id: 220075996

Degree: Bsc. (Hons) Computer System Engineering

Off Campus: ISMT College

Title: Android Mobile Development

Contents

1. Introduction

In today's fast-paced lifestyle, individuals often find themselves caught up in the relentless rhythm of daily life, leaving little room for reflection on significant events and associated expenses. The contemporary era places a high premium on tourism, with an increasing desire among people to embark on new and exciting adventures, exploring fresh destinations. While these vacations invariably provide enjoyable experiences, they can also become fleeting memories. As we return to our routines then we can calculate total expenses by looking at bought items. Consequently, when friends or family express an interest in traveling to the same destination and seek guidance, the ability to provide required items to assist them effectively.

2. Analysis

2.1 Various approaches to mobile app design and development

Mobile app development comes in various flavors. Native app development focuses on creating apps for specific platforms like iOS or Android, ensuring top-notch performance and user experience but requiring separate efforts for each platform. Cross-platform development, using tools like React Native and Flutter, allows apps to work smoothly on both iOS and Android with a single codebase. Progressive Web Apps, on the other hand, offer web-like experiences within browsers, accessible across platforms through web links. Choosing the right approach depends on factors like project complexity, budget, timeline, and target audience. Each approach has its pros and cons, emphasizing the need for careful consideration in app development decisions.

2.2 Operating System

An Operating System in smartphones and tablets is a critical software layer managing hardware resources, software apps, and user interactions. It enables seamless functionality, efficient resource allocation, and mobile device security. Mobile OSs like iOS, Android, and others offer user-friendly interfaces, robust security, and app support, optimizing performance and enhancing the user experience. (Lea, 2022) Android, from Google, powers a wide range of smart devices with customization options. iOS, exclusive to Apple devices, boasts a user-friendly interface, top-notch security, and seamless integration with Apple hardware.

Comparison of Android and iOS operating systems based on various aspects presented in a given table.

Aspect	Android	iOS
Company	Google	Apple
Open Source	Yes	No
Device Variety	Different range of mobile have android operating system	One and only found in Apple devices
App Ecosystem	Google Play Store and also found on third party stores	App Store
Security	Generally consider more vulnerable due to fragmentation	Strong emphasis on security and privacy
Updates	Android updates vary by company and may be delayed	Consistency update directly by Apple
User Interface	Diverse UI Design across different smart phone companies	Uniform Interface
Voice Assistance	Google Assistance	Siri
Cost	Android device are cheap and vary from different companies	iOS devices are typically premium priced
Programming Language	JAVA and KOTLIN	Swift and Objective-C

2.3 Different Programming Language

The primary programming languages for Android are Java and Kotlin. Java, a versatile and widely-used language, was traditionally the language of choice for Android app development. However, Kotlin, a more modern language with enhanced features and concise syntax, has gained popularity and is now officially supported for Android development. Kotlin offers improved developer productivity and safety. (Boushehrinejadmoradi, 2015) Apple's preferred languages are Swift and Objective-C. Swift is a relatively new and highly efficient language developed by Apple. Objective-C, while older, is still used for legacy projects and offers a robust, object-oriented programming paradigm.

2.4 Storage Design

When developing mobile applications for data storage, you have various database options at your disposal, which can be categorized as local or cloud-based solutions. Examples of these databases include SQLite-Room, SQLite, and Firebase. Notably, SQLite stands out as the world's most widely employed database engine due to its superior speed, self-contained nature, and exceptional reliability compared to alternative databases. It's important to note that SQLite operates as a local server in this context. In my android application, I am going to user Firebase database which is cloud based database provided by the Google.

2.5 Tools for development

Comparing the development tools for iOS and Android reveals some intriguing differences. While Android development once relied on Eclipse, the introduction of Google's Android Studio marked a significant shift. Android Studio has since become the standard, offering a versatile environment for app creation. Android's open development platform further empowers developers with a wide array of third-party apps and tools, fostering innovation and enhancing app functionality. In contrast, iOS app development centers on Xcode, described as the "core of the Apple programming experience." Although Apple's development platform is somewhat more constrained in terms of tool selection, it mandates the use of specific tools, limiting room for experimentation with novel concepts.

2.6 Security

In the realm of security, iOS boasts a strong advantage due to its restricted environment and the intricacy of downloading apps outside of the App Store. This tight control enables Apple to roll out periodic security updates, resulting in a relatively low occurrence of security issues. (Alenezi, 2017) Nevertheless, this level of constraint might be seen as limiting to some users. In contrast, Android offers monthly security updates, yet the challenge lies in the timely distribution by manufacturers. Consequently, a significant portion of Android devices often runs on outdated OS software, highlighting the trade-offs between security and flexibility in the Android ecosystem.

2.7 Storage

Android seamlessly integrates cloud services, like Google Drive, offering 15GB of free storage and flexible plans starting at $2 per month for 100GB or $10 monthly for 1TB. Android users also benefit from dedicated apps for Amazon Photos, OneDrive, and Dropbox. In contrast, iOS

provides iCloud with 5GB free storage and paid options: $1/month for 50GB, $3/month for 200GB, and $10/month for 1TB. iOS devices come with dedicated apps for Google Drive, Google Photos, Amazon Photos, OneDrive, and Dropbox, offering diverse cloud storage choices.

2.8 Benefits of native mobile development

Enhanced Performance: Native mobile development enables direct interaction with a device's hardware and software, resulting in superior speed and efficiency compared to cross-platform alternatives.

Immersive User Experience: Native apps deliver a cohesive and recognizable user interface by adhering to the specific design principles and behaviors of each platform, be it iOS or Android.

Harnessing Platform Capabilities: Developers can readily tap into platform-specific features and APIs, facilitating the creation of advanced functionalities such as push notifications, camera integration, and location-based services.

Heightened Security: Native apps benefit from the security measures furnished by platform providers, reducing the exposure to vulnerabilities and ensuring robust data protection.

Offline Capability: Native apps are proficient in functioning without an internet connection, ensuring a smoother user experience by storing and utilizing cached data and content. This feature is particularly advantageous in scenarios with limited or intermittent internet connectivity.

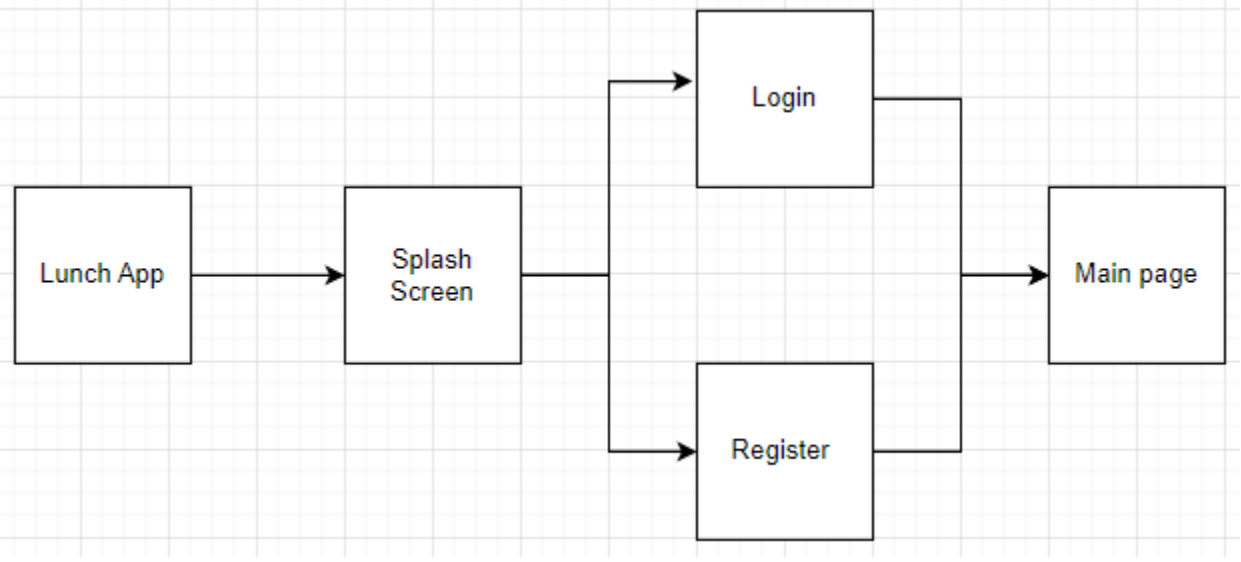
3. Design

In this section, we'll delve into the design of my prototype. This project adopts a functional design methodology, which assists in comprehending the project's design by breaking it down into modules that explain its flow, use cases, and implementation. Within this project, there are several modules, each with independent functionality, as well as other sub-modules for specific functions. All these elements are individually created, constructed, and then combined to build a fully functional application.

3.1 Screen Hierarchy

1. When you launch the app, it displays a splash screen featuring the app's name. This serves as an initial screen that appears briefly.

2. Following that, the login page appears if the user is not already logged in. If the user is already logged in, the main page or home page is shown. On the login page, users can navigate to the registration page if it's their first time using the app.
3. Once all the required fields on the registration page are filled out, the user is successfully registered within the app.
4. After logging in or registering, the user gains access to the main page. Here, users can view all the items they've added and can also add new items by clicking on the Floating Action Button.
5. When items are created, clicking on the card in the list allows you to see options for editing and delegating.
6. Clicking on the "Edit Item" button enables you to update item details and delete items.
7. By clicking on the "Delegate" button, you can send an SMS to anyone with the item details included in the message box.
8. On the main page, you can click on the three dots or the menu icon, which will then reveal a "Log Out" button. Clicking this button allows you to successfully log out from the app.



3.2 Wireframe

Here is a link of Figma wireframe design:

https://www.figma.com/file/VysxpIk5wG8U3naeNKSl2g/Holiday-Trip?type=design&node-id=0%3A1&mode=design&t=irDvqUfpqeAxeirG-1

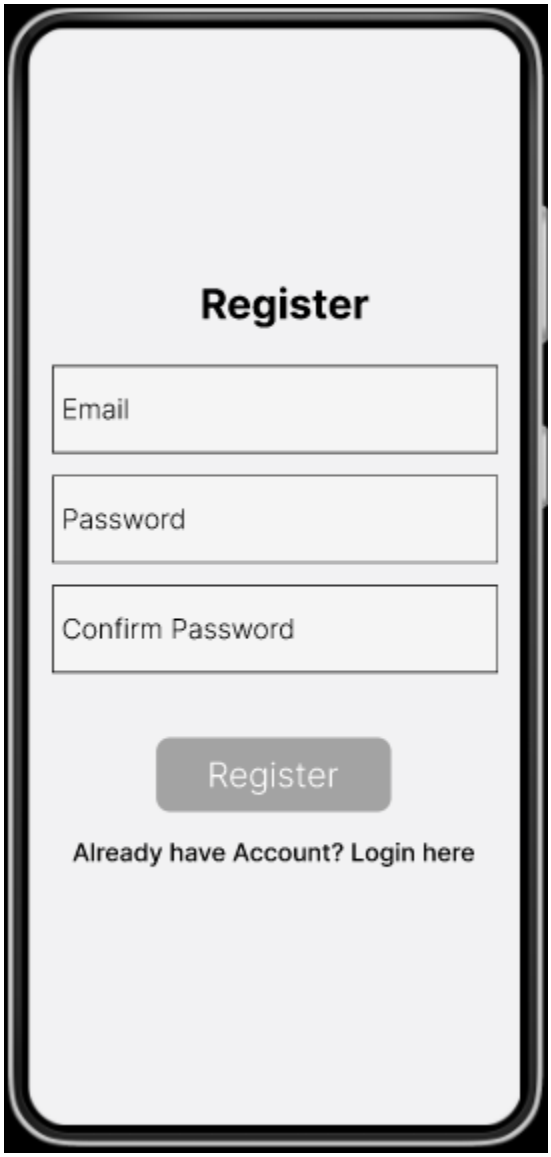
Splash Screen

- This is the first page of app.
- Splash screen show for some times likes 4 sec to show for branding and improving user experience during app lunch. Here is Holiday trip is a name of an app.

Login Screen

- This is a login screen. If user is already register then they can login by proving email and password.
- If a user is visiting first time then they can click on register here to register.
- After login user can navigate into main page.

Register Screen

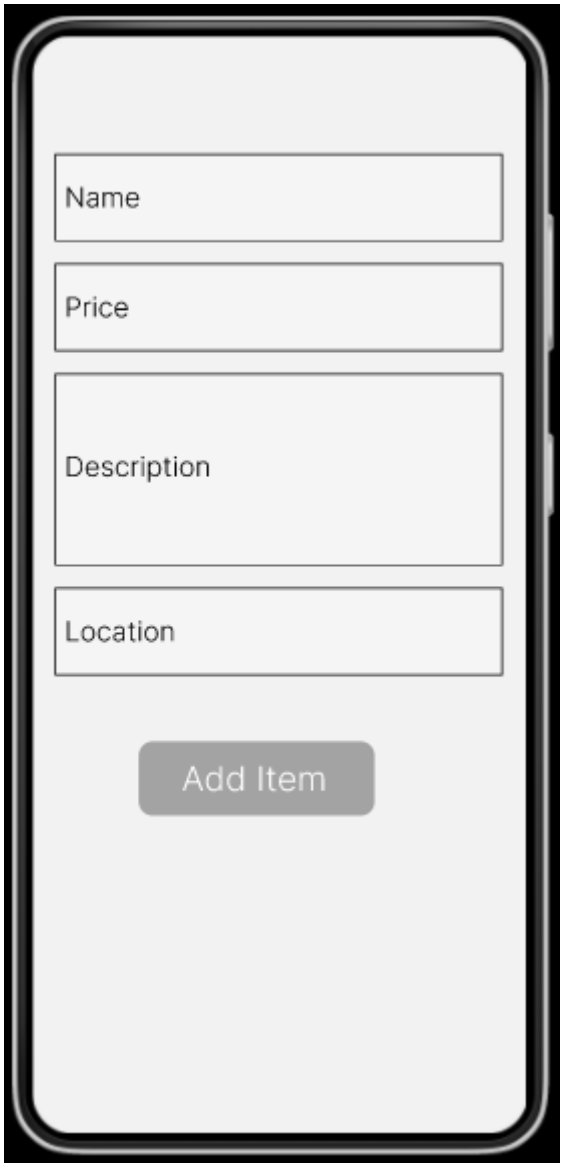


- This is a register screen. After clicking on register here is login page, user can see this register page.
- In the page user have to provide email, password and confirm password to register. Password and confirm password should be same.
- After registering user can navigate into main page.

Main page Screen

- This is a main page or home page. Here user can add new items and see items which were already added.
- To add new item they have to click on FAB button know as Floating Action Button.

Add Item Screen

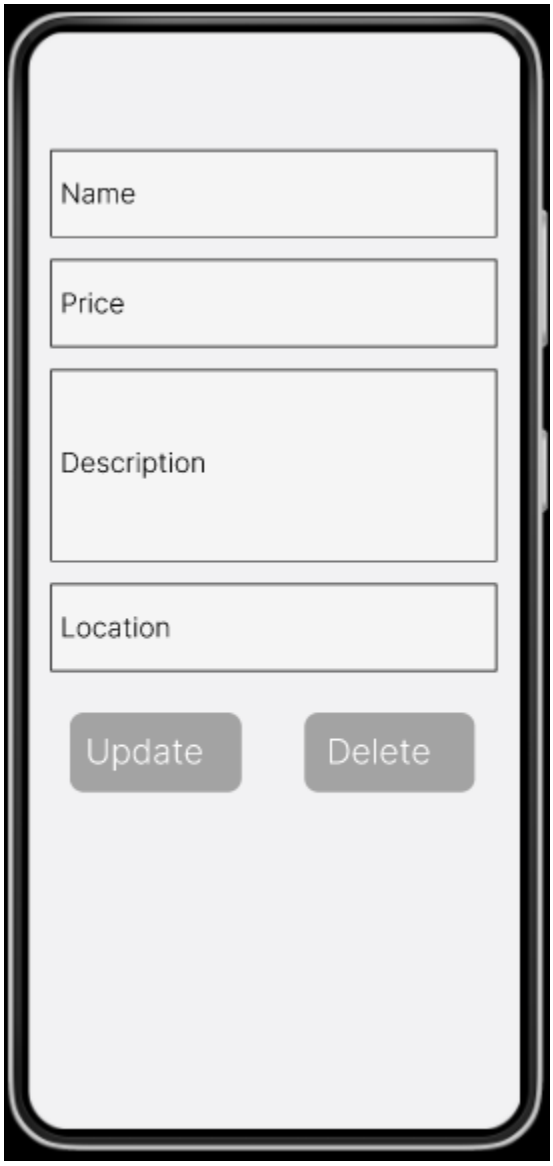


- This is add item screen. When user click on FAB button then they see this page. In this page user have to input item name, price, description and location then click on add item button.

Bottom Sheet Dialog Screen

- This is showed when user click on items. After clicking on items user can see bottom sheet dialog with edit item and delegate buttons.
- Edit item button navigate into a page where user can update item and delete it.
- Delegate button navigate into a page where user can send sms.

Update and Delete Screen



- This is update and delete page. In this page user can update items by changing name, price, description and location.
- User can also delete items by clicking on Delete button.

Delegate Message Screen

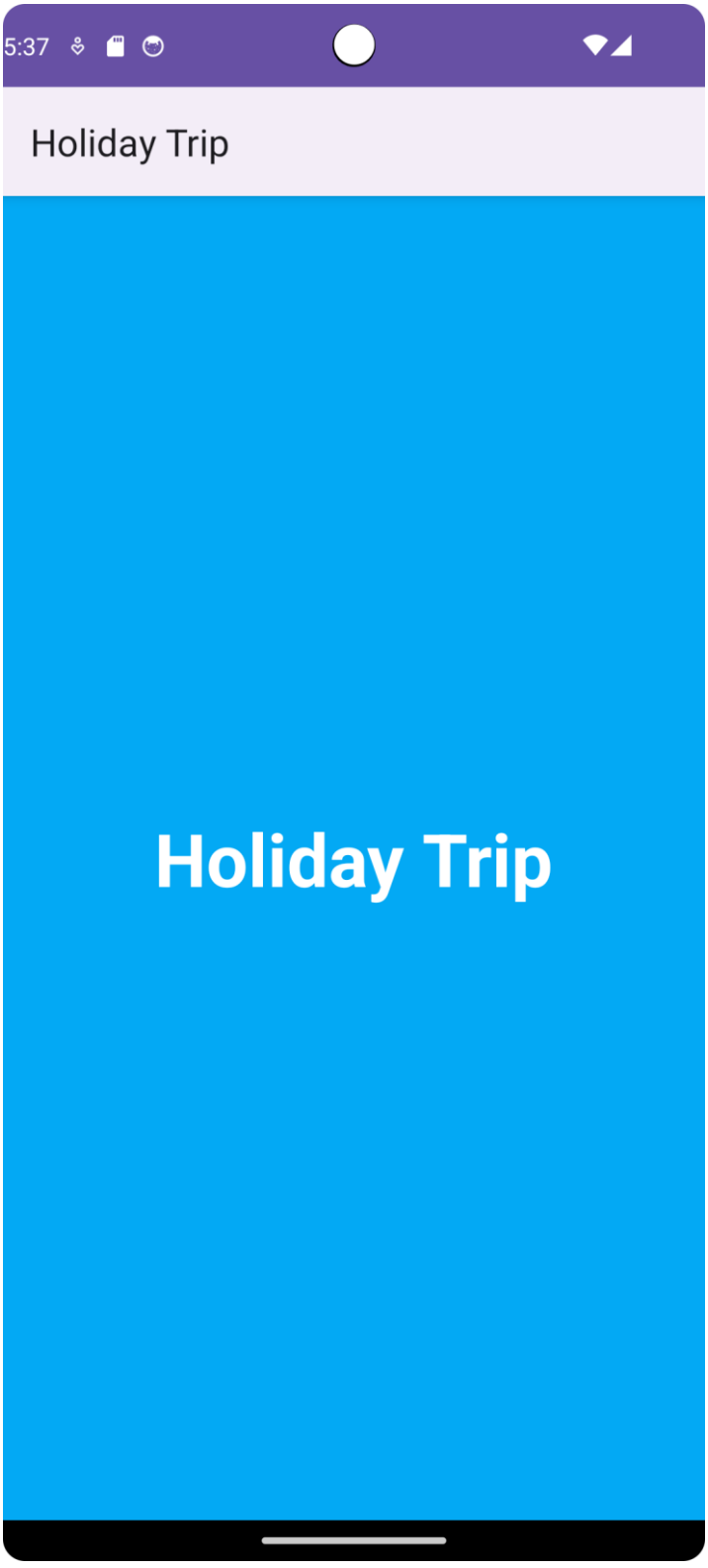
- This a Delegate SMS screen. In this page user have to provide phone number in which he want to send items details.
- After entering phone number user have to click on send SMS button to send SMS. Message is by default included (Details of the items).

Logout Menu Screen

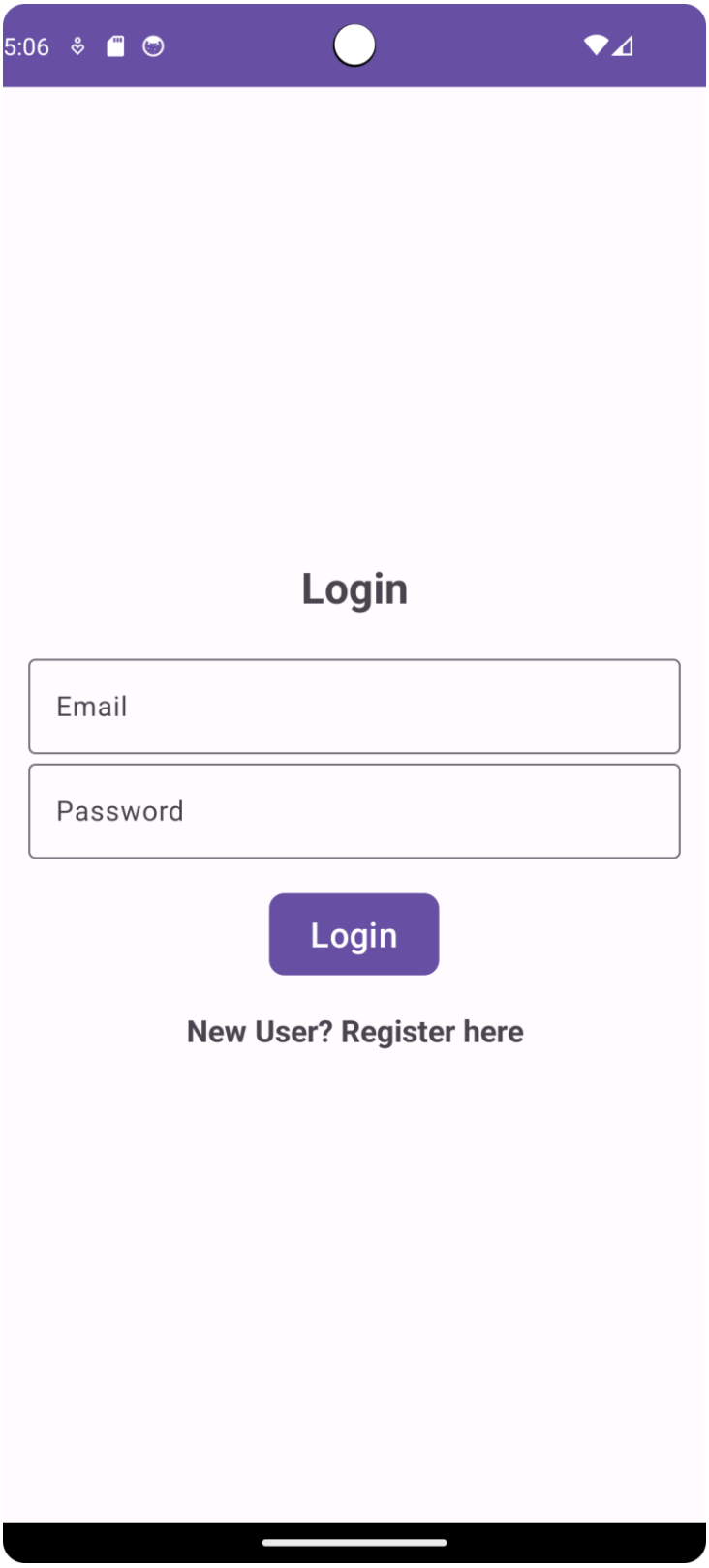
- This is a log out page. If user want to log out from the app they have to click on three dot in the top right corner.
- After that log out button appear then user have to click on that button to log out. By logout user end the current session to app.

3.3 Implementation of wireframe

Splash screen



Login Screen



Principles

The login page have several Material Design principles which are given below:

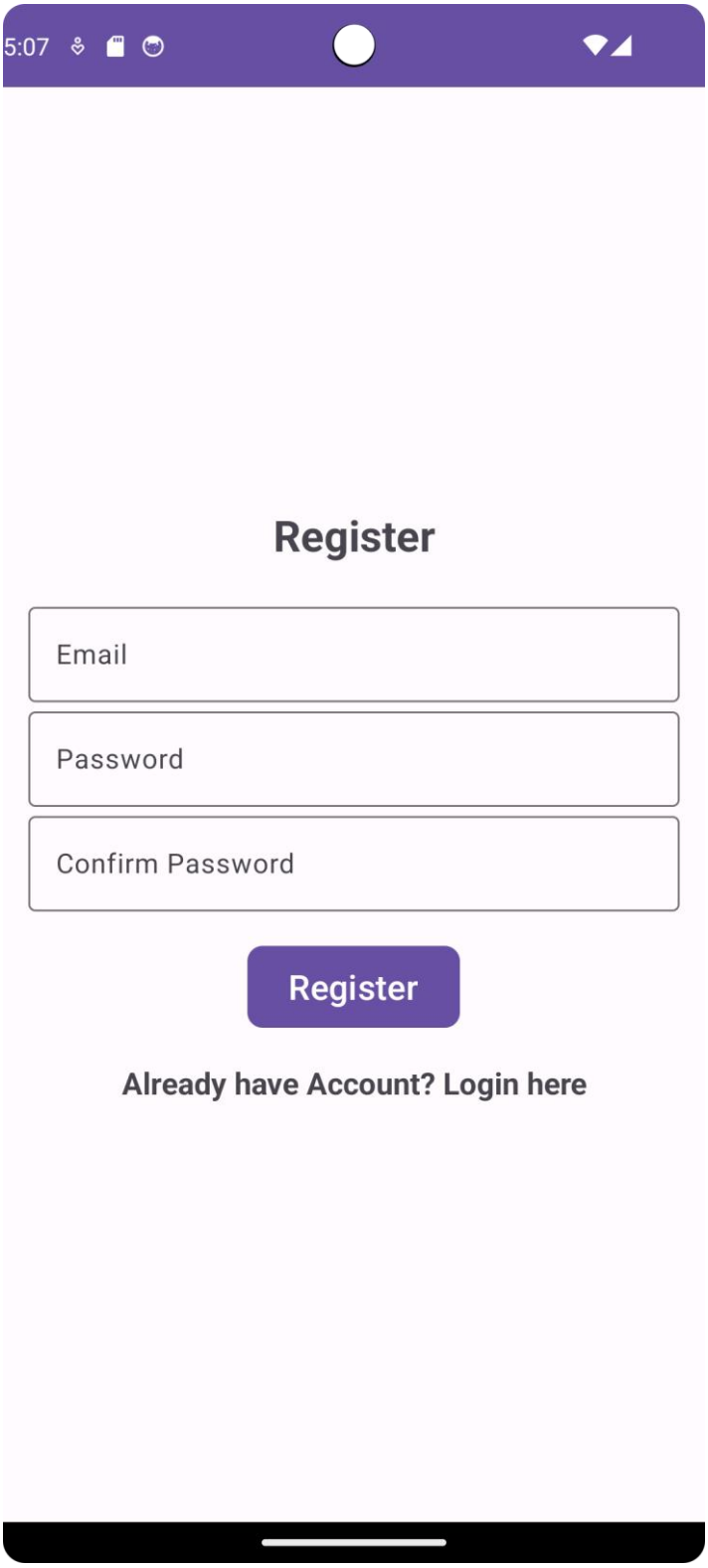
TextInputLayout: TextInputLayout is used for email and password input fields follows the Material Design metaphor of text fields. It provides a clear visual indication of the input area and adds floating labels that move above the input when a user interacts with it, which is a key aspect of Material Design.

Button Design: The login button, styled with a custom background, maintains the material metaphor by providing a tactile surface that reacts to touch feedback when clicked.

Typography: The use of typography for the "Email" and "Password" hints and the "New User? Register here" text adheres to Material Design's principles of readable and consistent text styles.

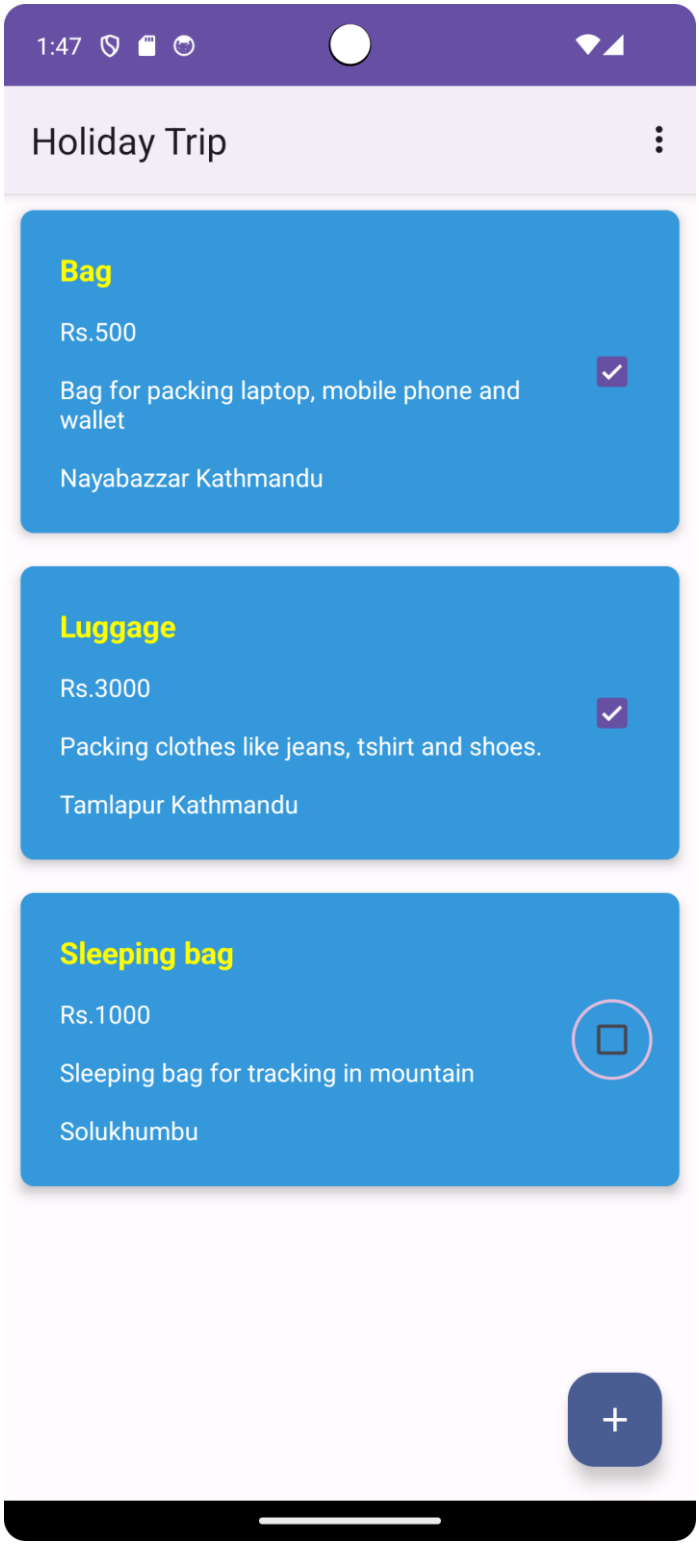
Spacing and Layout: The spacing and layout of elements, including margins and padding, align with Material Design's recommendations for creating visually balanced and easily scannable user interfaces.

Register Screen



In the register page same material design principles are used.

Main Screen



In the main page I have used consistence structure for the recycle view item layout. Here are other principle used is this page are:

1. Use of Checkbox

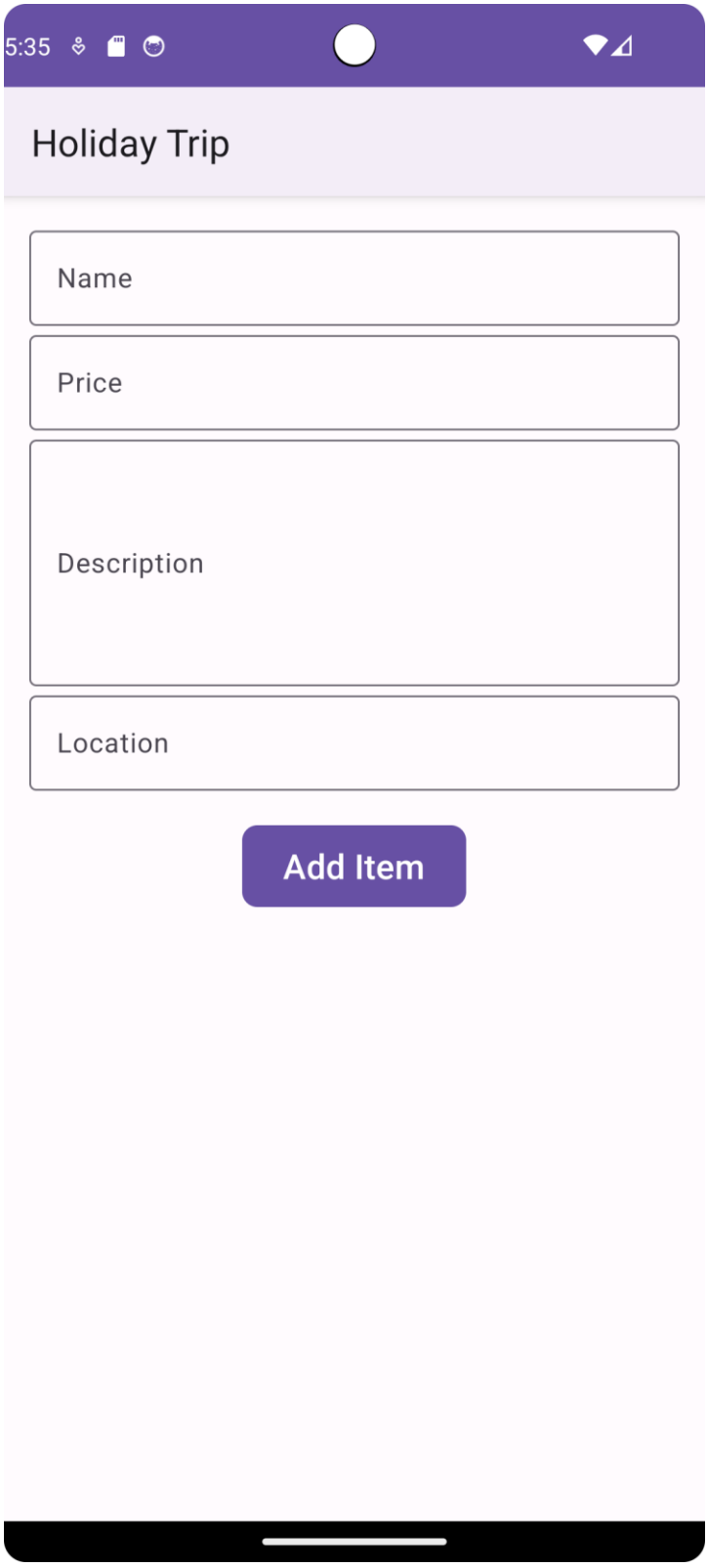
   Checkbox are included in each recycle view to add an items in purchased and vice versa.
2. Floating Action Button(FAB)

   FAB is used to add item in a menu list
3. Use of padding, margin and text style

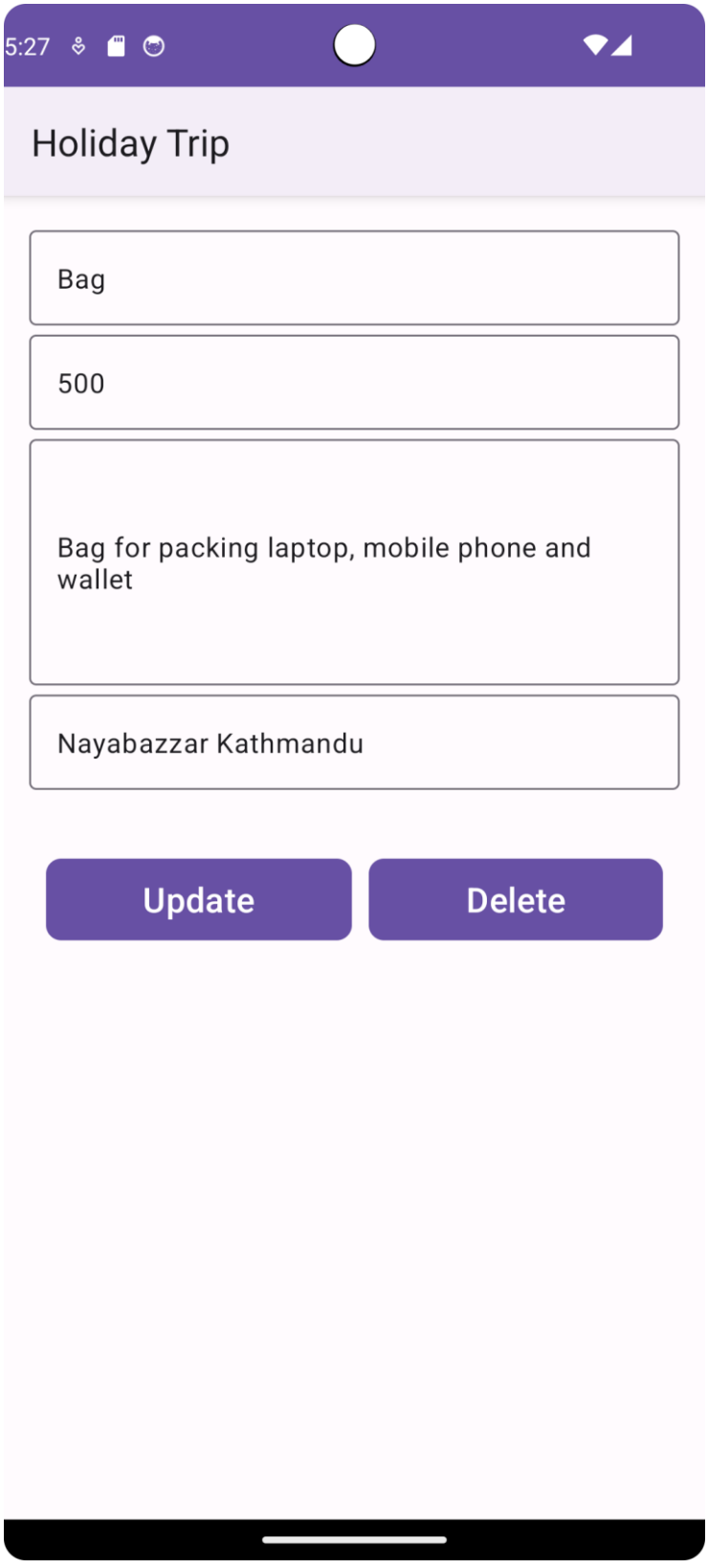
   Use of padding, margin and text style make items looks good and place properly in the main page.
4. Progress Indicator

   When data are not fetch from the firebase database for taking time to fetch it then progress bar or indicator appear.

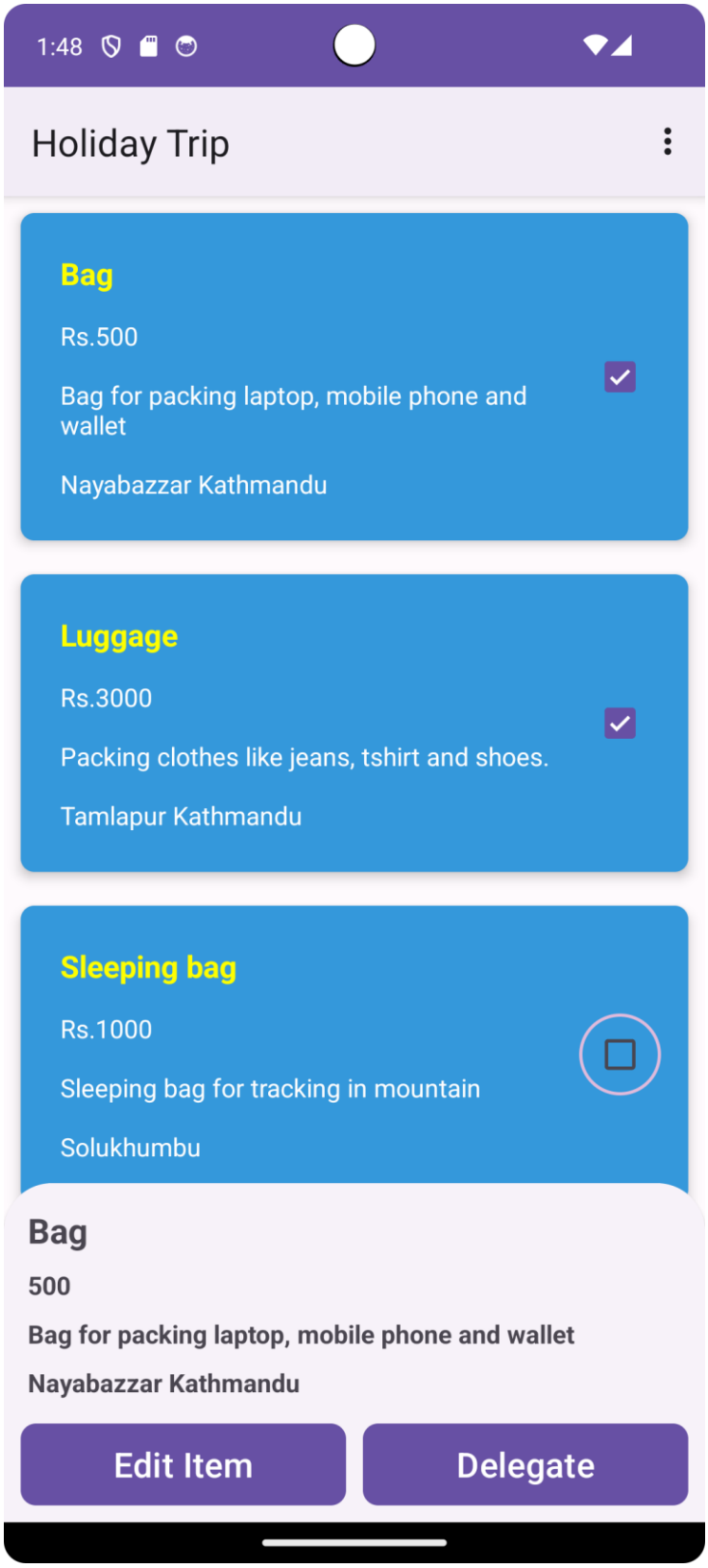
Add Screen



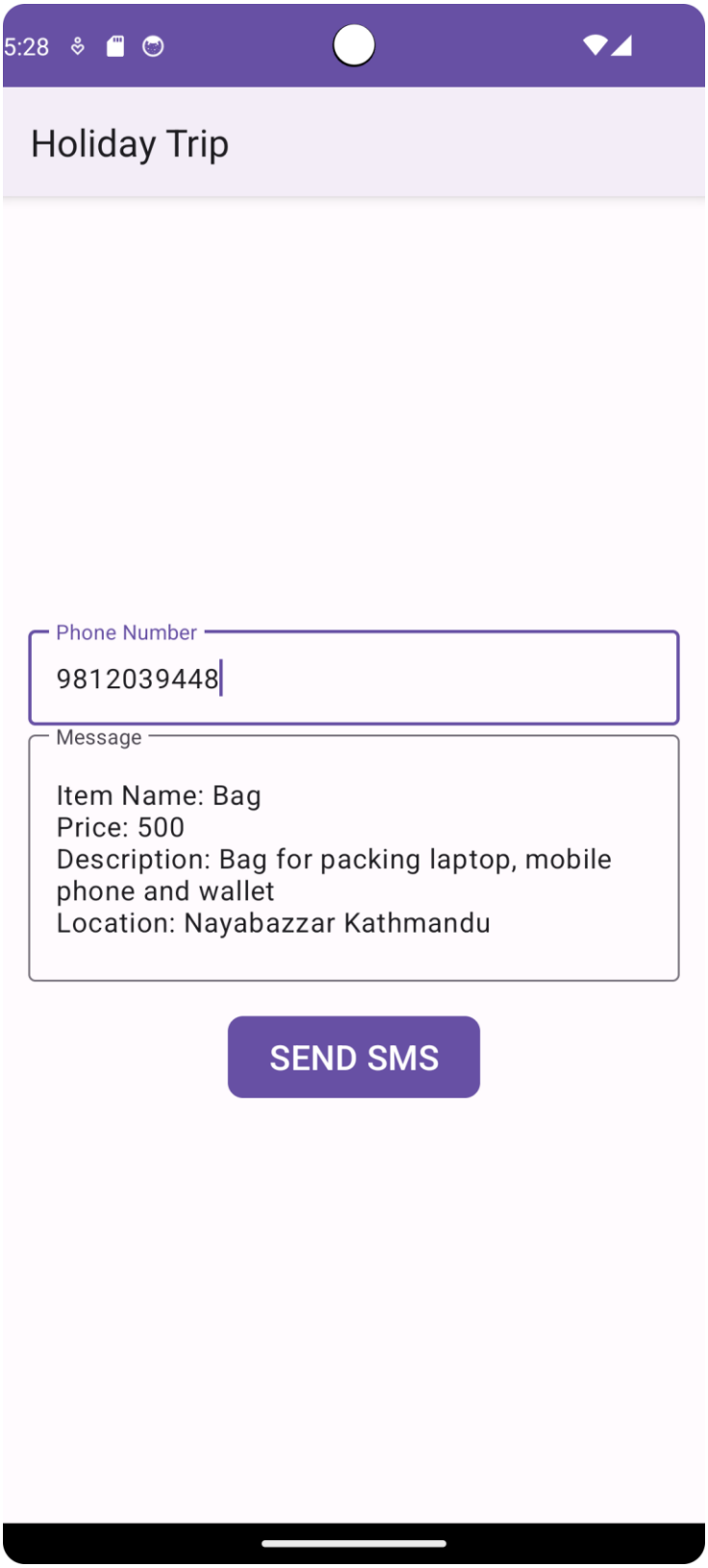
Update Screen



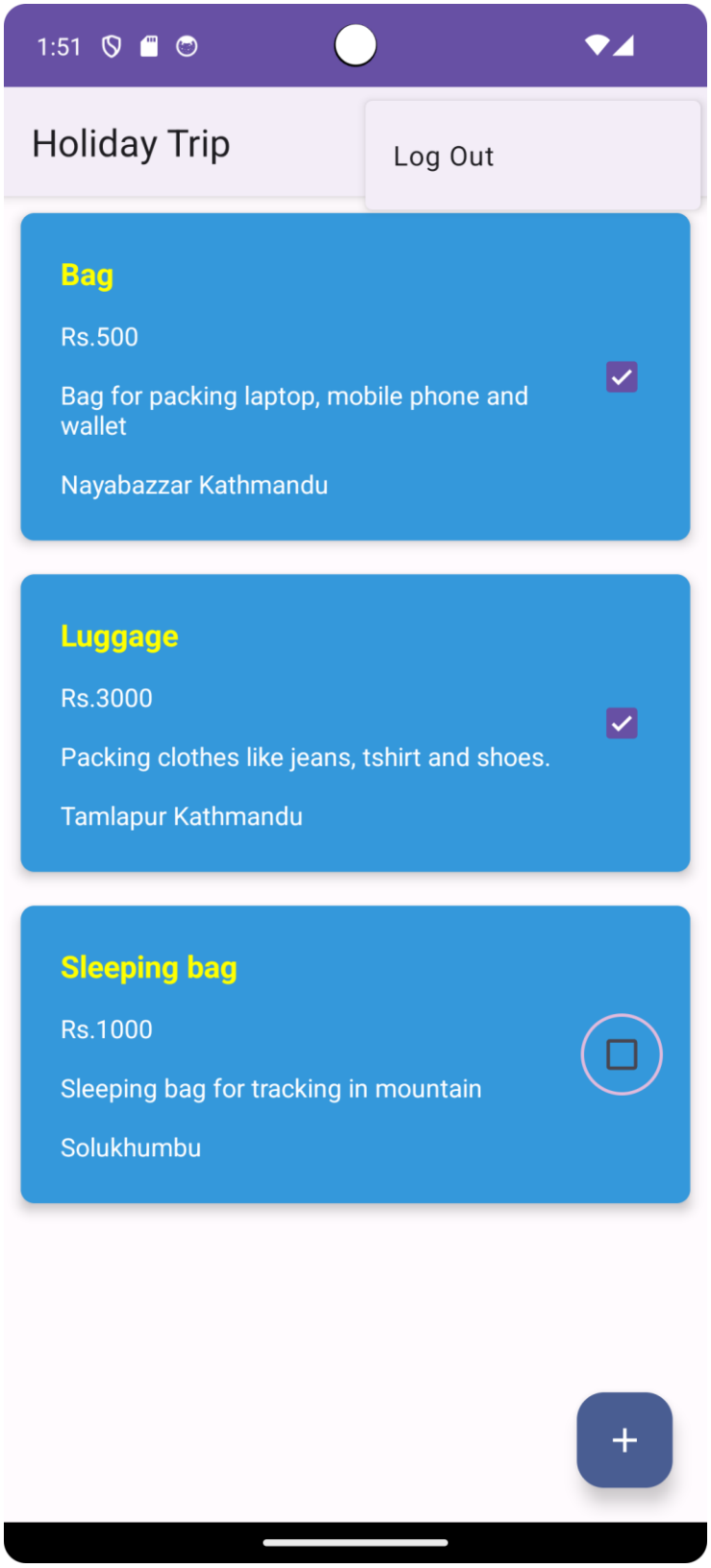
Bottom Sheet Dialog



Delegate SMS



Log Out Screen



IN Add, update, Delegate SMS, bottom sheet dialog and log out all used same material UI Principles which I were used in Login and Main Screen.

4. Functionality

This application's primary function is to collect item details from registered users who are planning trips. Users must register to use the app, and once registered, they can add, update, or delete item details and send SMS messages to friends with item information. Here are the description and code snipped of functionality.

Login Activity

The login process begins immediately after the splash screen. Users are prompted to input their email and password, which is a prerequisite for accessing the application. The login functionality is encapsulated within the "Login" file, where all the Java code responsible for this process resides. Meanwhile, the user interface for the login is defined in the "activity_login.xml" file, which contains all the XML code necessary for rendering the login screen.

For added security, user login credentials are securely stored in the Firebase database, and the passwords are encrypted to safeguard against potential security breaches. In cases where a user attempts to click the login button without entering their email and password, the system provides immediate feedback by displaying a toast message to remind the user to complete the required fields. To further illustrate the login page's appearance, you can refer to the screenshot provided in the wireframe implementation section above.

Code:

```
// Handle the login button click event
buttonLogin.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        progressBar.setVisibility(View.VISIBLE);
        String email, password;
        email = String.valueOf(editTextEmail.getText());
        password = String.valueOf(editTextPassword.getText());

        // Check if email and password fields are empty and show a toast message if they are
        if (TextUtils.isEmpty(email)) {...}
        if (TextUtils.isEmpty(password)) {...}

        // Authenticate user using Firebase authentication
        mAuth.signInWithEmailAndPassword(email, password)
                .addOnCompleteListener(new OnCompleteListener<AuthResult>() {
                    @Override
                    public void onComplete(@NonNull Task<AuthResult> task) {
                        progressBar.setVisibility(View.GONE);
                        // Check if login was successful and navigate to the main activity
                        if (task.isSuccessful()) {
                            Toast.makeText( context: Login.this,  text: "Login Successful",
                                    Toast.LENGTH_SHORT).show();
                            Intent intent = new Intent(getApplicationContext(), MainActivity.class);
                            startActivity(intent);
                            finish();
                        } else {
                            // Show a toast message if authentication fails
                            Toast.makeText( context: Login.this,  text: "Authentication failed.",
                                    Toast.LENGTH_SHORT).show();
                        }
                    }
                });
    }
});
```

Register Activity

The registration process follows a similar pattern to the login procedure in our app. After the initial splash screen, users are directed to a registration page where they are required to input their email, password, and confirm their password. The registration functionality is encapsulated within the "Register" file, which contains all the Java code responsible for handling the registration process. Meanwhile, the user interface for the registration page is defined in the "activity_register.xml" file, encompassing all the necessary XML code for rendering this page.

In alignment with security best practices, user registration details are securely stored in the Firebase database. Passwords are encrypted to fortify the application against potential security threats. Much like the login functionality, our system incorporates validation to ensure that users provide all the required information before proceeding. Should a user attempt to register without filling in the essential fields, a toast message promptly alerts them to complete the necessary information. To gain a visual understanding of the registration page's layout, you can reference the wireframe implementation section for a provided screenshot.

Code:

```
        // Handle the register button click event
        buttonReg.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                progressBar.setVisibility(View.VISIBLE);

                String email, password, confirmPassword;
                email = String.valueOf(editTextEmail.getText());
                password = String.valueOf(editTextPassword.getText());
                confirmPassword = String.valueOf(editTextConfirmPassword.getText());


                // Check if email, password, and confirm password fields are empty and show a toast message if they are
                if (TextUtils.isEmpty(email)) {...}
                if (TextUtils.isEmpty(password)) {...}
                if (TextUtils.isEmpty(confirmPassword)) {...}
                // Check if password and confirm password match and show a toast message if they don't
                if(!password.equals(confirmPassword)){...}

                // Create a new user account using Firebase authentication
                mAuth.createUserWithEmailAndPassword(email, password)
                        .addOnCompleteListener(new OnCompleteListener<AuthResult>() {
                            @Override
                            public void onComplete(@NonNull Task<AuthResult> task) {
                                progressBar.setVisibility(View.GONE);
                                if (task.isSuccessful()) {
                                    Toast.makeText( context: Register.this,  text: "Account created successfully",
                                            Toast.LENGTH_SHORT).show();
                                    Intent intent = new Intent(getApplicationContext(), Login.class);
                                    startActivity(intent);
                                    finish();
                                } else {
                                    // Show a toast message if user account creation fails
                                    Toast.makeText( context: Register.this,  text: "Authentication failed.",
                                            Toast.LENGTH_SHORT).show();
                                }
                            }
```

Add Activity

Within this activity, users can seamlessly incorporate items into their lists by simply tapping the Floating Action Button (FAB) on the main page. To include an item, users are prompted to furnish essential details such as the item name, price, description, and the intended purchase location. This "Add" feature stands as one of the central functions of our application, streamlining the process of item management. All the intricate particulars of each item are efficiently stored within a real-time Firebase database. The dedicated activity for this action goes by the name "AddItem," and to ensure a seamless user experience, we've crafted the corresponding user interface in the form of the "activity_add_item.xml" file.

Code:

```
77        addItemButton.setOnClickListener(new View.OnClickListener() {
78            @Override
79            public void onClick(View view) {
80                String planName = name.getText().toString().trim();
81                String planPrice = price.getText().toString().trim();
82                String planDesc = description.getText().toString().trim();
83                String planLocation = location.getText().toString().trim();
84
85
86                if (planName.isEmpty() || planPrice.isEmpty() || planDesc.isEmpty() || planLocation.isEmpty()) {
87                    // At least one field is empty, display an error message
88                    Toast.makeText( context: AddItem.this,  text: "Please fill in all fields", Toast.LENGTH_SHORT).show();
89                } else {
90                    progressBar.setVisibility(View.VISIBLE);
91                    planID = planName;
92                    ItemRVModel itemRVModel = new ItemRVModel(planName, planPrice, planDesc, planLocation, planID, isPurchased);
93                    databaseReference.child(planID).setValue(itemRVModel)
94                            .addOnSuccessListener(new OnSuccessListener<Void>() {
95                                @Override
96                                public void onSuccess(Void aVoid) {
97                                    progressBar.setVisibility(View.GONE);
98                                    Toast.makeText( context: AddItem.this,  text: "Plan Added", Toast.LENGTH_SHORT).show();
99                                    startActivity(new Intent( packageContext: AddItem.this, MainActivity.class));
100                                }
101                            })
102                            .addOnFailureListener(new OnFailureListener() {
                                no usages
103                                @Override
104                                public void onFailure(@NonNull Exception e) {
105                                    progressBar.setVisibility(View.GONE);
106                                    Toast.makeText( context: AddItem.this,  text: "Error: " + e.getMessage(), Toast.LENGTH_SHORT).show();
107                                }
108                            });
109                }
110            }
111        });
```

Update Activity

Update activity and add activity are similar but only different is changing existence values. EditItem is a name of activity and layout name is activity_edit_item.xml. After clicking update button it change it edited value in database.

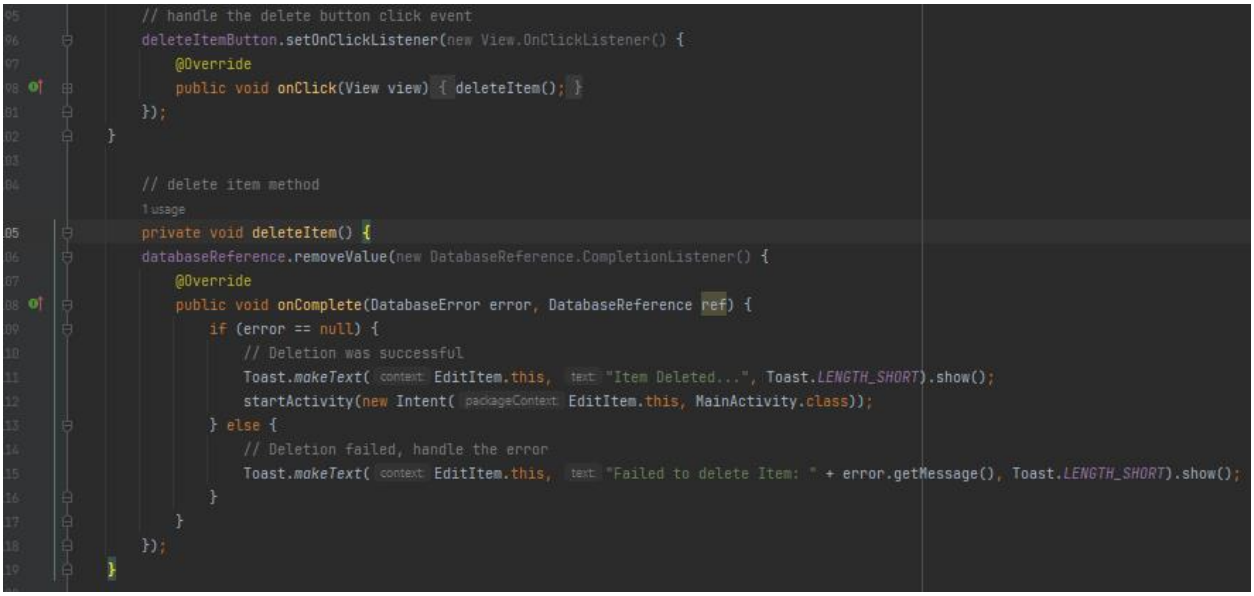
Code:

```
updateItemButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        progressBar.setVisibility(View.VISIBLE);
        String planName = name.getText().toString();
        String planPrice = price.getText().toString();
        String planDesc = description.getText().toString();
        String planLocation = location.getText().toString();

        Map<String,Object> map = new HashMap<>();
        map.put( k "planName",planName);
        map.put( k "planPrice",planPrice);
        map.put( k "planDesc",planDesc);
        map.put( k "planLocation",planLocation);
        map.put( k "planID",planID);

        databaseReference.addValueEventListener(new ValueEventListener() {
            2 usages
            @Override
            public void onDataChange(@NonNull DataSnapshot snapshot) {
                progressBar.setVisibility(View.GONE);
                databaseReference.updateChildren(map);
                Toast.makeText( context EditItem.this,  text "Plan Updated", Toast.LENGTH_SHORT).show();
                startActivity(new Intent( packageContext EditItem.this, MainActivity.class));
                databaseReference.removeEventListener(this);

            }

            @Override
            public void onCancelled(@NonNull DatabaseError error) {

                Toast.makeText( context EditItem.this,  text "Fail to update course..", Toast.LENGTH_SHORT).show();

            }
        });
    }
});
```

Delete Functionality

The process of initiating the "Delete" functionality starts when the user clicks the delete button within a list. Upon the successful completion of the delete operation, a toast message is displayed, indicating that the item has been deleted. Following user confirmation, the Android application communicates with the Firebase Realtime Database to remove the item from the database as well.

Code:



```
        // handle the delete button click event
        deleteItemButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) { deleteItem(); }
        });
    }

    // delete item method
    1 usage
    private void deleteItem() {
        databaseReference.removeValue(new DatabaseReference.CompletionListener() {
            @Override
            public void onComplete(DatabaseError error, DatabaseReference ref) {
                if (error == null) {
                    // Deletion was successful
                    Toast.makeText( context: EditItem.this,  text: "Item Deleted...", Toast.LENGTH_SHORT).show();
                    startActivity(new Intent( packageContext: EditItem.this, MainActivity.class));
                } else {
                    // Deletion failed, handle the error
                    Toast.makeText( context: EditItem.this,  text: "Failed to delete Item: " + error.getMessage(), Toast.LENGTH_SHORT).show();
                }
            }
        });
    }
```

Send SMS

Send SMS functionality or delegate enables the user to send an SMS message containing item information to a specified phone number after requesting and obtaining the necessary SMS permission. It provides feedback on the status of the SMS sending process through toast messages and navigation back to the main activity. The app initializes UI components and retrieves item information. Upon user click, it checks and requests SMS permissions, sends an SMS with item details to a specified number, and displays success or error messages.

Code:

```
14 usages
@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);

    // check condition
    if (requestCode == 100 && grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
        sendSMS();
    } else {
        // when permission denied
        Toast.makeText( context: this,  text: "Permission Denied!", Toast.LENGTH_SHORT).show();
    }

}

2 usages
private void sendSMS() {

    String number = smsNum.getText().toString();
    String message = smsMsg.getText().toString();

    // check string is empty or not

    if (!number.isEmpty() && !message.isEmpty()) {
        SmsManager smsManager = SmsManager.getDefault();

        smsManager.sendTextMessage(number,  scAddress: null, message,  sentIntent: null,  deliveryIntent: null);

        Toast.makeText( context: this,  text: "SMS send successfully", Toast.LENGTH_SHORT).show();
        Intent i = new Intent( packageContext: sms_activity.this, MainActivity.class);
        startActivity(i);
    } else {
        Toast.makeText( context: this,  text: "Please enter phone number.", Toast.LENGTH_SHORT).show();
    }
}
}
```

Logout

Logout functionality involves a straightforward process in holiday trip application. When the user triggers the logout button that is appear when user click on menu, the app communicates with Firebase Authentication to sign the user out. Firebase Authentication manages user sessions, so logging out typically means invalidating the user's authentication token or session. The Firebase SDK provides a simple method, such as mAuth.signout() to handle this. Once the user is signed out, after that user redirect into to login page.

Code:

```
4 usages
@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {
    int id = item.getItemId();
    if (id == R.id.idLogOut) {
        // displaying a toast message on user logged out inside on click.
        Toast.makeText(getApplicationContext(), text: "User Logged Out", Toast.LENGTH_LONG).show();
        // on below line we are signing out our user.
        mAuth.signOut();
        // on below line we are opening our login activity.
        Intent i = new Intent( packageContext: MainActivity.this, Login.class);
        startActivity(i);
        this.finish();
        return true;
    } else {
        return super.onOptionsItemSelected(item);
    }

}
```

5. Testing

Testing involves evaluating errors and defects in mobile apps or software to ensure they meet specific requirements and function as intended. It helps identify bugs and verifies whether the app performs as expected. For instance, I'm planning to conduct a mobile usability test for my holiday trip app.

Why is it important?

Conducting a mobile usability test is crucial to ensure the effectiveness and user-friendliness of a mobile app. This testing allows developers to gain valuable insights into how users interact with the app, identify any potential roadblocks or frustrations users might encounter, and collect feedback on overall user experience. (Forrester, 2023) By observing real users navigating the app and gathering their feedback, as a developer, I can improve design and functionality, resulting in an app that is intuitive, easy to use, and aligns well with user expectations. A positive user experience is key to app success, as it fosters user satisfaction, encourages usage, and can lead to higher app ratings, increased user retention, and ultimately, a successful app in the competitive mobile market.

How do you go through it?

The methodology involves a series of steps outlined as follows:

1. Establishing objectives

2. Compiling a list of tasks

3. Developing test materials

4. Selecting suitable testers—in this instance, the tester will be me.

5. Acquiring the necessary devices; I tested the app on Samsung A5.

6. Conducting the actual evaluation.

Test Documentation

S.N	**Action**	**Inputs**	**Predictable Output**	**Real Output**	**Result**
1	Start App		Holiday Trip splash screen	Holiday Trip splash screen	Pass
2	Enter login button with providing email and password.	No email and password	Please enter email	Please enter email	Pass
3	Enter wrong email and password	Wrong email and password	Authentication failed	Authentication failed	Pass
4	Enter email, password and confirm password correct	Email: govinda11@gmail.com Password: ******** Confirm Password: *******	Register successful	Register successful	Pass

5	Go to login directly	Click on new user? login here	Login page appear	Login page appear	Pass
6	Go to register directly	Click on new user? Register here	Register page appear	Register page appear	Pass
7	Add new items	Click on Fab button and add item form appear. Fill out all form and click add Item button	Item successfully add in database and show in home page	Item successfully added in to database and showed in home page.	Pass
8	Update Item	Click on Edit item and update item page appear, update necessary field and click update button	Item successfully updated and show updated item in home page	Tem successfully updated and show updated item in home page	Pass
9	Delete Item	Click on Edit item button and delete page appear then click on delete button.	Item deleted from database and disappear item from home page	Item deleted from database and disappear item from home page	Pass
10	Delegate SMS	Click on Item card then click on Delegate button and Delegate SMS page open enter phone number and click on SEND SMS button.	SMS send successfully	SMS send successfully	Pass

11	Add to purchase	Click checkbox to add item into purchase list	Item added successfully in purchase list	Item added successfully in purchase list	Pass
12	Logout	Click three dot or menu in top to see log out button and click on logout button	Logout successfully and show login page	Logout successfully and show login page	Pass

6. Evaluation

The "Holiday Trip" Android app was thoroughly evaluated based on its core features, including a splash screen, user authentication (login and register), item management (add, update, delete), and the user interface. The splash screen functioned seamlessly, providing a smooth transition into the main interface and setting a positive initial impression. The user authentication process worked effectively, ensuring secure access to the app through login and register functionalities with proper validation and error handling. The item management features, encompassing add, update, and delete actions, were intuitive and displayed relevant toast messages upon successful completion. These features facilitated easy item management for trip planning and organization. Additionally, the app showcased an intuitive and visually appealing user interface, contributing to a seamless user experience. Recommendations for further enhancement include the addition of features like in-app purchases, location-based services, and social sharing functionalities to enhance user engagement during trips. Continuous monitoring of user feedback and iterative improvements will play a crucial role in optimizing the app's performance and overall user satisfaction. In conclusion, the "Holiday Trip" app effectively fulfills its intended functionality, providing travelers with a user-friendly platform for efficient trip planning and management, accompanied by a visually pleasing and intuitive user interface.

7. Conclusion

In conclusion, the "Holiday Trip" Android app has been successfully developed and tested, incorporating essential features such as a splash screen, login, register, item management (add, display, update, delete), and convenient in-app communication via SMS. The toast messages enhance user experience and provide feedback during actions. To further enhance the app,

considering potential UI/UX improvements and performance optimizations would be beneficial for an even smoother user interaction.

References

Alenezi, M. a. A. I., 2017. *Abusing Android permissions: A security perspective.* [Online]
Available at: https://ieeexplore.ieee.org/document/8257772
[Accessed 23 September 2023].

Boushehrinejadmoradi, N. a. G. V. a. N. S. a. I. L., 2015. *Testing Cross-Platform Mobile App Development Frameworks (T).* [Online]
Available at: https://ieeexplore.ieee.org/document/7372032
[Accessed 28 September 2023].

Forrester, A. a. B. E. a. D. A. a. T. J., 2023. *How to Build Android Apps with Kotlin: A practical guide to developing, testing, and publishing your first Android apps.* [Online]
[Accessed 1 October 2023].

Lea, R. a. Y. Y. a. I. J.-I., 2022. *Adaptive operating system design using reflection.* [Online]
Available at: https://ieeexplore.ieee.org/document/513462
[Accessed 5 October 2023].